

LAPORAN PRAKTIKUM
STRUKTUR DATA
“Shell, Merge, dan Quick Sort”

disusun Oleh:

Sofian Arba’i

2511533029

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur kehadiran Allah SWT karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan Laporan Praktikum pekan 8 dengan judul “Shell Sort, Merge Sort, dan Quick Sort” ini dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban kegiatan praktikum serta sebagai sarana untuk meningkatkan pemahaman mengenai penggunaan algoritma pengurutan data dalam struktur data.

Dalam praktikum pekan 8, dipelajari dasar-dasar algoritma sorting lanjutan, yaitu teknik yang digunakan untuk mengurutkan data ke dalam urutan tertentu secara lebih efisien. Praktikum ini membahas tiga algoritma pengurutan, yaitu Shell Sort, Merge Sort, dan Quick Sort.

Penyusunan laporan ini tidak terlepas dari bantuan, bimbingan, dan dukungan berbagai pihak. Oleh karena itu, saya ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan ilmu, arahan, serta bimbingannya.
2. Asisten praktikum yang telah membimbing dan memberikan penjelasan selama kegiatan praktikum berlangsung.

Saya meminta maaf jika masih terdapat kesalahan pada penulisan laporan ini. Oleh karena itu, saya menerima kritik dan saran yang membangun demi perbaikan di masa mendatang.

Padang, 25 Mei 2026

Sofian Arba'i

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN.....	3
2.1 Pengertian Shell, Merge, dan Quick Sort	3
2.1.1 Kelebihan dan Kekurangan Shell, Merge, dan Quick Sort	4
2.2 Implementasi Shell, Merge, dan Quick Sort.....	5
2.2.1 Class ShellSort_2511533029.....	5
2.2.2 Class MergeSort_2511533029	7
2.2.3 Class QuickSort_2511533029	9
2.2.4 Class BubleSortGUI_2511533029	11
BAB III PENUTUP	18
3.1 Kesimpulan	18
3.2 Saran	18
DAFTAR PUSTAKA.....	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan struktur data, proses pengurutan data (sorting) merupakan salah satu teknik yang sangat penting untuk mempermudah pengelolaan dan pencarian data. Algoritma sorting digunakan untuk menyusun data ke dalam urutan tertentu, seperti dari kecil ke besar atau sebaliknya, sehingga data menjadi lebih teratur dan mudah diproses. Seiring berkembangnya jumlah data yang semakin besar, diperlukan algoritma pengurutan yang lebih efisien dan cepat.

Pada praktikum pekan 8 ini dipelajari beberapa algoritma sorting lanjutan, yaitu Shell Sort, Merge Sort, dan Quick Sort. Ketiga algoritma tersebut memiliki cara kerja dan tingkat efisiensi yang berbeda dalam proses pengurutan data. Shell Sort menggunakan konsep gap untuk mempercepat proses sorting, Merge Sort menggunakan metode divide and conquer dengan membagi dan menggabungkan data, sedangkan Quick Sort melakukan pengurutan berdasarkan pivot untuk membagi data menjadi beberapa bagian.

Selain implementasi algoritma sorting, praktikum ini juga membahas visualisasi proses Bubble Sort menggunakan Java Swing berbasis GUI. Dengan adanya visualisasi tersebut, proses pengurutan data dapat dipahami dengan lebih mudah karena pengguna dapat melihat langkah-langkah sorting secara langsung. Oleh karena itu, praktikum ini dilakukan untuk memahami konsep, implementasi, serta cara kerja berbagai algoritma sorting dalam pemrograman Java.

1.2 Tujuan

Tujuan yang diharapkan pada praktikum pekan 8 tentang algoritma sorting shell, merge dan quick serta GUI bubble sort yaitu:

1. Mempelajari cara kerja algoritma Shell Sort, Merge Sort, dan Quick Sort.
2. Mengimplementasikan algoritma sorting menggunakan bahasa pemrograman Java.
3. Mengetahui perbedaan efisiensi dan proses kerja dari masing-masing algoritma sorting.
4. Memahami visualisasi proses Bubble Sort menggunakan GUI Java Swing.

1.3 Manfaat

Manfaat yang didapatkan setelah melakukan praktikum pekan 8 tentang algoritma sorting shell, merge dan quick serta GUI bubble sort yaitu:

1. Melatih kemampuan dalam mengimplementasikan algoritma sorting menggunakan Java.
2. Membantu memahami proses pengurutan data secara lebih efisien.
3. Mempermudah pengguna memahami proses sorting melalui visualisasi GUI.
4. Menjadi dasar pembelajaran untuk pengembangan program yang membutuhkan pengolahan data terurut.

BAB II

PEMBAHASAN

2.1 Pengertian Shell, Merge, dan Quick Sort

Dalam dunia pemrograman, sangat penting untuk memiliki algoritma penyortiran efisien yang memungkinkan kita mengatur sejumlah besar data dengan cepat dan akurat. Berikut pengertian dari Shell, Merge, dan Quick Sort.

1. Shell Sort

Metode Shell Sort, juga dikenal sebagai Shell Sort, adalah algoritma pengurutan yang dikembangkan oleh Donald Shell pada tahun 1959. Algoritma ini didasarkan pada gagasan membagi daftar asli menjadi subdaftar yang lebih kecil dan mengurutkannya secara independen. Lalu, gabungkan subdaftar yang telah diurutkan ini untuk memperoleh daftar akhir yang telah diurutkan. Metode Shell Sort merupakan penyempurnaan pada algoritma penyisipan langsung karena mengurangi jumlah perbandingan dan pergeseran yang diperlukan.

2. Merge Sort

Algoritma Merge Sort adalah salah satu metode pengurutan data yang berbasis perbandingan dan memanfaatkan teknik “divide and conquer” atau “bagi dan taklukkan”. Metode ini efisien untuk mengurutkan kumpulan data dengan ukuran besar.

Pada dasarnya, algoritma Merge Sort memecah daftar data menjadi bagian-bagian kecil, mengurutkan masing-masing bagian tersebut secara terpisah, dan kemudian menggabungkannya kembali menjadi satu kesatuan dalam urutan yang benar.

Algoritma Merge Sort ini sangat efektif karena dalam setiap langkahnya, ukuran data yang harus diurutkan berkurang menjadi setengahnya. Oleh karena itu, kompleksitas waktu algoritma ini adalah $O(n \log n)$, di mana “n” adalah jumlah elemen data yang akan diurutkan. Ini menjadikan Merge Sort pilihan yang cocok untuk mengurutkan data dalam skala besar, terutama ketika efisiensi waktu menjadi pertimbangan utama

3. Quick Sort

Quick Sort adalah algoritma pengurutan yang sering digunakan. Algoritma ini bekerja dengan membagi set data menjadi dua bagian, yaitu bagian yang lebih kecil dan bagian yang lebih besar dari elemen pivot. Kemudian, urutan dimulai dengan pengurutan pada bagian yang lebih kecil dan kemudian mengurutkan bagian yang lebih besar.

Quick Sort pertama kali diperkenalkan oleh Tony Hoare pada tahun 1960-an. Algoritma ini sangat efektif dalam mengurutkan set data yang besar dan sering dianggap sebagai salah satu algoritma pengurutan yang paling efektif.

2.1.1 Kelebihan dan Kekurangan Shell, Merge, dan Quick Sort

Berikut beberapa kelebihan dan kekurangan dari masing-masing algoritma sorting.

1. Shell Sort

Kelebihan dari shell (Implementasi yang mudah, Efisien untuk elemen dengan jarak yang lebih kecil). Kekurangan (Tidak efisien untuk ukuran array yang besar, Tidak efisien untuk elemen yang tersebar luas).

2. Merge Sort

Kelebihan (Stabil dan sangat cepat untuk data besar).
Kekurangan (Membutuhkan memori tambahan, Kompleksitas Penggabungan, Tidak Optimal untuk Data Kecil).

3. Quick Sort

Kelebihan (Mudah diimplementasikan, Fleksibel, Efisiensi).
Kekurangan (Proses implementasinya lebih rumit, Menggunakan rekursi yang dapat menyebabkan penggunaan stack, Pemilihan pivot yang kurang baik dapat membuat pembagian data tidak seimbang).

2.2 Implementasi Shell, Merge, dan Quick Sort

2.2.1 Class ShellSort_2511533029

Kode program pertama pekan 8 yaitu kode program algoritma Shell Sort yang dapat digunakan untuk mengurutkan data secara lebih efisien dibandingkan metode sorting sederhana seperti Bubble Sort. Berikut kode program shell sort.


```

1 package pekan8_2511533029;
2
3 public class ShellSort_2511533029 {
4
5     public static void shellSort_3029 (int [] A_3029) {
6         int n_3029 = A_3029.length;
7         int gap_3029 = n_3029/2;
8         while (gap_3029 > 0) {
9             for (int i = gap_3029; i < n_3029; i++) {
10                 int temp_3029 = A_3029[i];
11                 int j_3029 = i;
12                 while (j_3029 >= gap_3029 && A_3029[j_3029-gap_3029] > temp_3029) {
13                     A_3029 [j_3029] = A_3029[j_3029 - gap_3029];
14                     j_3029 = j_3029-gap_3029;
15                 }
16                 A_3029[j_3029] = temp_3029;
17             }
18             gap_3029 = gap_3029 / 2;
19         }
20     }
21 }
22
23 public static void main (String [] args) {
24     int []data_3029 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
25
26     System.out.print("sebelum: ");
27     printArray_3029(data_3029);
28
29     shellSort_3029 (data_3029);
30
31     System.out.print("Sesudah shell sort: ");
32     printArray_3029(data_3029);
33 }
34
35 public static void printArray_3029 (int[] arr_3029) {
36     for (int i : arr_3029) System.out.print(i + " ");
37     System.out.println();
38 }

```

Kode program 2.1

Kode program 2.1 merupakan implementasi algoritma Shell Sort menggunakan bahasa pemrograman Java. Program dibuat dalam kelas ShellSort_2511533029 yang berfungsi untuk mengurutkan data integer dari nilai terkecil ke terbesar. Metode utama yang digunakan adalah shellSort_3029(int[] A_3029). Pada metode ini, proses pengurutan dilakukan dengan menentukan nilai jarak (gap) yang awalnya bernilai setengah dari panjang array. Selanjutnya, data dibandingkan dan dipindahkan berdasarkan jarak tersebut hingga posisi yang sesuai ditemukan. Setelah satu proses selesai, nilai gap akan terus dibagi dua sampai bernilai 0, yang menandakan proses pengurutan telah selesai.

Pada method main, program mendeklarasikan array data_3029 yang berisi beberapa angka acak. Program kemudian menampilkan data sebelum diurutkan menggunakan method printArray_3029, lalu memanggil method shellSort_3029 untuk melakukan proses sorting.

Setelah proses selesai, hasil pengurutan ditampilkan kembali sehingga pengguna dapat melihat perbedaan data sebelum dan sesudah diurutkan. Method `printArray_3029` sendiri digunakan untuk mencetak seluruh isi array ke layar.

2.2.2 Class MergeSort_2511533029

Kode program selanjutnya adalah algoritma Merge Sort yang mampu mengurutkan data secara efisien dengan teknik pembagian dan penggabungan array sehingga menghasilkan data yang terurut dari kecil ke besar. Berikut kode program merge sort.

```

1. package pekan8_2511533029;
2.
3. public class MergeSort_2511533029 {
4.     void merge(int arr_3029[], int l_3029, int m_3029, int
      r_3029) {
5.
6.         // Find sizes of two subarrays to be merged
7.         int n1_3029 = m_3029 - l_3029 + 1;
8.         int n2_3029 = r_3029 - m_3029;
9.
10.        /* Create temp arrays */
11.        int L_3029[] = new int[n1_3029];
12.        int R_3029[] = new int[n2_3029];
13.
14.        /* Copy data to temp arrays */
15.        for (int i_3029 = 0; i_3029 < n1_3029;
          ++i_3029)
16.            L_3029[i_3029] = arr_3029[l_3029 +
          i_3029];
17.
18.        for (int j_3029 = 0; j_3029 < n2_3029;
          ++j_3029)
19.            R_3029[j_3029] = arr_3029[m_3029 + 1 +
          j_3029];
20.
21.        int i_3029 = 0, j_3029 = 0;
22.
23.        // Initial index of merged subarray array
24.        int k_3029 = l_3029;
25.
26.        while (i_3029 < n1_3029 && j_3029 < n2_3029)
27.        {
28.            if (L_3029[i_3029] <= R_3029[j_3029]) {
29.                arr_3029[k_3029] = L_3029[i_3029];
                i_3029++;
            }

```

```

30.         } else {
31.             arr_3029[k_3029] = R_3029[j_3029];
32.             j_3029++;
33.         }
34.         k_3029++;
35.     }
36.
37.     /* Copy remaining elements of L[] if any */
38.     while (i_3029 < n1_3029) {
39.         arr_3029[k_3029] = L_3029[i_3029];
40.         i_3029++;
41.         k_3029++;
42.     }
43.
44.     /* Copy remaining elements of R[] if any */
45.     while (j_3029 < n2_3029) {
46.         arr_3029[k_3029] = R_3029[j_3029];
47.         j_3029++;
48.         k_3029++;
49.     }
50. }
51. void sort_3029(int arr_3029[], int l_3029, int
r_3029) {
52.     if (l_3029 < r_3029) {
53.
54.         // Find the middle point
55.         int m_3029 = (l_3029 + r_3029) / 2;
56.
57.         // Sort first and second halves
58.         sort_3029(arr_3029, l_3029, m_3029);
59.         sort_3029(arr_3029, m_3029 + 1, r_3029);
60.
61.         // Merge the sorted halves
62.         merge(arr_3029, l_3029, m_3029, r_3029);
63.     }
64. }
65.
66. /* A utility function to print array of size n
*/
67. static void printArray_3029(int arr_3029[]) {
68.     int n_3029 = arr_3029.length;
69.
70.     for (int i_3029 = 0; i_3029 < n_3029;
++i_3029)
71.         System.out.print(arr_3029[i_3029] + "
");
72.
73.     System.out.println();
74. }
75.
76. public static void main(String args_3029[]) {
77.
78.     int arr_3029[] = {12, 11, 13, 5, 6, 7};
79.

```

```

80.         System.out.println("Sebelum terurut");
81.         printArray_3029(arr_3029);
82.
83.         MergeSort_2511533029 ob_3029 = new
MergeSort_2511533029();
84.         ob_3029.sort_3029(arr_3029, 0,
arr_3029.length - 1);
85.
86.         System.out.println("\nSesudah Terurut
menggunakan Merge Sort");
87.         printArray_3029(arr_3029);
88.     }
89. }

```

Program di atas merupakan implementasi algoritma Merge Sort menggunakan bahasa pemrograman Java untuk mengurutkan data integer dari kecil ke besar. Proses pengurutan dilakukan dengan metode divide and conquer, yaitu membagi array menjadi dua bagian secara rekursif melalui method `sort_3029()`, kemudian menggabungkan kembali bagian yang telah terurut menggunakan method `merge()`. Pada proses merge, program membandingkan elemen dari dua array sementara lalu menyusunnya kembali ke array utama secara berurutan. Method `main` digunakan untuk menampilkan data sebelum dan sesudah proses sorting, sedangkan `printArray_3029()` berfungsi mencetak isi array ke layar.

2.2.3 Class QuickSort_2511533029

Kode program selanjutnya yaitu algoritma Quick Sort dapat mengurutkan data secara cepat dan efisien dengan teknik pembagian array berdasarkan pivot. Berikut kode program quick sort.

```

1. package pekan8_2511533029;
2.
3. public class QuickSort_2511533029 {
4.     static void swap_3029 (int [] arr_3029, int i_3029,
int j_3029) {
5.         int temp_3029 = arr_3029[i_3029];
6.         arr_3029 [i_3029] = arr_3029 [j_3029];
7.         arr_3029 [j_3029] = temp_3029;
8.
9.     }

```

```

10. // metode tambahan untuk mengatur pivot menggunakan
    median of three
11. static void medianOfThree_3029 (int [] arr_3029, int
    low_3029, int high_3029 ) {
12.     int mid_3029 = low_3029 + (high_3029 -
        low_3029) / 2;
13.
14.     // urutkan elemen low , mid dan high
15.     if (arr_3029 [low_3029] > arr_3029[mid_3029])
    {
16.         swap_3029 (arr_3029, low_3029,
            mid_3029);
17.     }
18.     if (arr_3029 [low_3029] > arr_3029
        [high_3029]) {
19.         swap_3029 (arr_3029, low_3029,
            high_3029);
20.     }
21.     if (arr_3029 [mid_3029] > arr_3029
        [high_3029]) {
22.         swap_3029 (arr_3029, mid_3029,
            high_3029);
23.     }
24.     swap_3029 (arr_3029, mid_3029, high_3029);
25. }
26. static int paritition_3029 (int [] arr_3029, int
    low_3029, int high_3029) {
27.     //panggil fungsi median of three sebelum
    menentukan pivot
28.     int pivot_3029 = arr_3029[high_3029]; //
    sekarang arr[high] sudah berisi nilai median
29.     int i_3029 = (low_3029 - 1);
30.
31.     for (int j_3029 = low_3029; j_3029 <=
        high_3029 -1; j_3029++) {
32.         // jika elemen saat ini lebih kecil
        dari atau sama dengan pivot
33.         if (arr_3029[j_3029] < pivot_3029) {
34.             // increment indeks elemen yang
            lebih kecil
35.             i_3029++;
36.             swap_3029 (arr_3029, i_3029,
                j_3029);
37.         }
38.     }
39.     swap_3029 (arr_3029, i_3029 +1, high_3029);
40.     return (i_3029 +1);
41. }
42. static void quickSort_3029 (int [] arr_3029, int
    low_3029, int high_3029) {
43.     if (low_3029 < high_3029) {
44.         int pi_3029 = paritition_3029(arr_3029,
            low_3029, high_3029);

```

```

45.         quickSort_3029 (arr_3029, low_3029,
46.         pi_3029 - 1);
47.         quickSort_3029 (arr_3029, pi_3029+1,
48.         high_3029);
49.     }
50.     public static void printArr_3029 (int [] arr_3029) {
51.         for (int i_3029 = 0; i_3029 <
52.         arr_3029.length; i_3029++) {
53.             System.out.print(arr_3029 [i_3029] + "
54.             ");
55.         }
56.         System.out.println();
57.     }
58.     public static void main (String [] args) {
59.         int[] arr_3029 = {10, 7, 8, 9, 1, 5};
60.         int N_3029 = arr_3029.length;
61.         System.out.print("Data sebelum dirurutkan:
62.         ");
63.         printArr_3029 (arr_3029);
64.         quickSort_3029 (arr_3029, 0, N_3029 - 1);
65.         System.out.print("Data terurut quikSort: ");
66.         printArr_3029 (arr_3029);
67.     }
68. }

```

Program di atas merupakan implementasi algoritma Quick Sort menggunakan bahasa pemrograman Java untuk mengurutkan data integer dari kecil ke besar. Program menggunakan metode divide and conquer dengan memilih pivot sebagai acuan pembagian data. Pemilihan pivot dilakukan menggunakan metode median of three melalui method `medianOfThree_3029()` agar proses sorting lebih optimal. Method `paritition_3029()` berfungsi membagi array berdasarkan pivot, yaitu menempatkan elemen yang lebih kecil di sebelah kiri dan elemen yang lebih besar di sebelah kanan. Selanjutnya, method `quickSort_3029()` melakukan proses pengurutan secara rekursif hingga seluruh data tersusun rapi. Method `main` digunakan untuk menampilkan data sebelum dan sesudah proses sorting, sedangkan `printArr_3029()` berfungsi mencetak isi array ke layar.

2.2.4 Class BubleSortGUI_2511533029

Kode program terakhir pekan 8 yaitu membuat kode program GUI, yaitu seperti aplikasi simulasi untuk algoritma bubble sort. Berikut kode program untuk Buble sort GUI.

```

1.  package pekan8_2511533029;
2.
3.  import java.awt.BorderLayout;
4.  import java.awt.Color;
5.  import java.awt.Dimension;
6.  import java.awt.FlowLayout;
7.  import java.awt.Font;
8.
9.  import javax.swing.*;
10. import javax.swing.border.EmptyBorder;
11.
12. public class BubleSortGUI_2511533029 extends JFrame {
13.     private static final long serialVersionUID = 1L;
14.     private int[] array_3029;
15.     private JLabel[] labelArray_3029;
16.     private JButton stepButton_3029, resetButton_3029,
17.     setButton_3029;
18.     private JTextField inputField_3029;
19.     private JPanel panelArray_3029;
20.     private JTextArea stepArea_3029;
21.     private int i_3029 = 1, j_3029;
22.     private boolean sorting_3029 = false;
23.     private int stepCount_3029 = 1;
24.
25.     /**
26.      * Create the frame.
27.      */
28.     public BubleSortGUI_2511533029() {
29.         setTitle("Insertion Sort Langkah per Langkah");
30.         setSize(750, 400);
31.         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
32.         setLocationRelativeTo(null);
33.         setLayout(new BorderLayout());
34.
35.         // Panel input
36.         JPanel inputPanel_3029 = new JPanel(new
37.         FlowLayout());
38.         inputField_3029 = new JTextField(30);
39.         setButton_3029 = new JButton("Set Array");
40.         inputPanel_3029.add(new JLabel("Masukkan angka
41.         (pisahkan dengan koma):"));
42.         inputPanel_3029.add(inputField_3029);
43.         inputPanel_3029.add(setButton_3029);
44.
45.         // Panel array visual
46.         panelArray_3029 = new JPanel();
47.         panelArray_3029.setLayout(new FlowLayout());

```

```

46.         // Panel kontrol
47.         JPanel controlPanel_3029 = new JPanel();
48.         stepButton_3029 = new JButton("Langkah
Selanjutnya");
49.         resetButton_3029 = new JButton("Reset");
50.         stepButton_3029.setEnabled(false);
51.         controlPanel_3029.add(stepButton_3029);
52.         controlPanel_3029.add(resetButton_3029);
53.
54.         // Area teks untuk log langkah-langkah
55.         stepArea_3029 = new JTextArea(8, 60);
56.         stepArea_3029.setEditable(false);
57.         stepArea_3029.setFont(new Font("Monospaced",
Font.PLAIN, 14));
58.         JScrollPane scrollPane_3029 = new
JScrollPane(stepArea_3029);
59.
60.         // Tambahkan panel ke frame
61.         add(inputPanel_3029, BorderLayout.NORTH);
62.         add(panelArray_3029, BorderLayout.CENTER);
63.         add(controlPanel_3029, BorderLayout.SOUTH);
64.         add(scrollPane_3029, BorderLayout.EAST);
65.
66.         // Event Set Array
67.         setButton_3029.addActionListener(e ->
setArrayFromInput_3029());
68.
69.         // Event Langkah Selanjutnya
70.         stepButton_3029.addActionListener(e ->
performStep_3029());
71.
72.         // Event reset
73.         resetButton_3029.addActionListener(e ->
reset_3029());
74.     }
75.
76.     private void setArrayFromInput_3029() {
77.         String text_3029 =
inputField_3029.getText().trim();
78.
79.         if (text_3029.isEmpty())
80.             return;
81.
82.         String[] parts_3029 = text_3029.split(",");
83.         array_3029 = new int[parts_3029.length];
84.
85.         try {
86.             for (int k_3029 = 0; k_3029 <
parts_3029.length; k_3029++) {
87.                 array_3029[k_3029] =
88.                 Integer.parseInt(parts_3029[k_3029].trim());
89.             }
90.         } catch (NumberFormatException e_3029) {

```



```

91.         JOptionPane.showMessageDialog(this,
92.             "Masukkan hanya angka yang
dipisahkan koma!",
93.             "Error",
94.             JOptionPane.ERROR_MESSAGE);
95.         return;
96.     }
97.     i_3029 = 0;
98.     j_3029 = 0;
99.     stepCount_3029 = 1;
100.    sorting_3029 = true;
101.    stepButton_3029.setEnabled(true);
102.    stepArea_3029.setText("");
103.    panelArray_3029.removeAll();
104.    labelArray_3029 = new JLabel[array_3029.length];
105.
106.    for (int k_3029 = 0; k_3029 < array_3029.length;
k_3029++) {
107.        labelArray_3029[k_3029] =
108.            new
JLabel(String.valueOf(array_3029[k_3029]));
109.        labelArray_3029[k_3029].setFont(new
Font("Arial", Font.BOLD, 24));
110.        labelArray_3029[k_3029].setOpaque(true);
111.
labelArray_3029[k_3029].setBackground(Color.WHITE);
112.        labelArray_3029[k_3029].setBorder(
113.            BorderFactory.createLineBorder(Color.BLACK));
114.        labelArray_3029[k_3029].setPreferredSize(
115.            new Dimension(50, 50));
116.
117.        labelArray_3029[k_3029].setHorizontalAlignment(
118.            SwingConstants.CENTER);
119.
panelArray_3029.add(labelArray_3029[k_3029]);
120.    }
121.    panelArray_3029.revalidate();
122.    panelArray_3029.repaint();
123.    }
124.
125.    private void performStep_3029() {
126.        if (!sorting_3029 || i_3029 >= array_3029.length
- 1) {
127.            sorting_3029 = false;
128.            stepButton_3029.setEnabled(false);
129.
130.            JOptionPane.showMessageDialog(this, "Sorting
selesai!");
131.            return;
132.        }
133.        resetHighlights_3029();

```

```

134.         StringBuilder stepLog_3029 = new
           StringBuilder();
135.
136.         labelArray_3029[j_3029].setBackground(Color.CYAN);
           labelArray_3029[j_3029 +
137.         1].setBackground(Color.CYAN);
138.
           if (array_3029[j_3029] > array_3029[j_3029 + 1])
139.         {
140.             // Swap
           int temp_3029 = array_3029[j_3029];
141.
           array_3029[j_3029] = array_3029[j_3029 + 1];
142.
           array_3029[j_3029 + 1] = temp_3029;
143.
144.         labelArray_3029[j_3029].setBackground(Color.RED);
           labelArray_3029[j_3029 +
145.         1].setBackground(Color.RED);
146.
           stepLog_3029.append("Langkah
147.           ").append(stepCount_3029).append(": Menukar elemen ke-
           ").append(j_3029).append(" ")
148.
149.           .append(array_3029[j_3029 + 1]).append(" dengan
           ke-").append(j_3029 + 1).append(" ")
150.
           .append(array_3029[j_3029]).append("\n");
151.
           } else {
152.
           stepLog_3029.append("Langkah
153.           ").append(stepCount_3029).append(": Tidak ada
           pertukaran antara ke-")
154.
           .append(j_3029).append(" dan ke-
           ").append(j_3029 + 1).append("\n");
155.
           }
156.
           stepLog_3029.append("Hasil: ")
157.
           .append(arrayToString_3029(array_3029))
           .append("\n\n");
158.
           stepArea_3029.append(stepLog_3029.toString());
159.
           updateLabels_3029();
           j_3029++;
160.
           if (j_3029 >= array_3029.length - i_3029 - 1) {
           j_3029 = 0;
161.
           i_3029++;
162.
           }
           stepCount_3029++;
163.
           if (i_3029 >= array_3029.length - 1) {
           sorting_3029 = false;
164.
           stepButton_3029.setEnabled(false);
165.

```

```

173.         JOptionPane.showMessageDialog(this, "Sorting
selesai!");
174.     }
175. }
176.
177.     private void updateLabels_3029() {
178.         for (int k_3029 = 0; k_3029 < array_3029.length;
k_3029++) {
179.             labelArray_3029[k_3029]
180.                 .setText(String.valueOf(array_3029[k
_3029]));
181.         }
182.     }
183.
184.     private void resetHighlights_3029() {
185.         for (JLabel label_3029 : labelArray_3029) {
186.             label_3029.setBackground(Color.WHITE);
187.         }
188.     }
189.
190.     private void reset_3029() {
191.         inputField_3029.setText("");
192.
193.         panelArray_3029.removeAll();
194.         panelArray_3029.revalidate();
195.         panelArray_3029.repaint();
196.         stepArea_3029.setText("");
197.         stepButton_3029.setEnabled(false);
198.         sorting_3029 = false;
199.         i_3029 = 0;
200.         j_3029 = 0;
201.         stepCount_3029 = 1;
202.     }
203.
204.     private String arrayToString_3029(int[] arr_3029) {
205.         StringBuilder sb_3029 = new StringBuilder();
206.
207.         for (int k_3029 = 0; k_3029 < arr_3029.length;
k_3029++) {
208.             sb_3029.append(arr_3029[k_3029]);
209.
210.             if (k_3029 < arr_3029.length - 1)
211.                 sb_3029.append(", ");
212.         }
213.         return sb_3029.toString();
214.     }
215.
216.     public static void main(String[] args_3029) {
217.         SwingUtilities.invokeLater(() -> {
218.             BubleSortGUI_2511533029 gui_3029 =
219.                 new BubleSortGUI_2511533029();
220.
221.             gui_3029.setVisible(true);
222.         });

```

```
223.     }  
224. }
```

Program di atas merupakan implementasi algoritma Bubble Sort berbasis GUI menggunakan bahasa pemrograman Java Swing. Program dibuat dalam kelas BubleSortGUI_2511533029 yang berfungsi untuk mengurutkan data angka dari kecil ke besar sekaligus menampilkan proses sorting secara visual. Antarmuka program terdiri dari input angka, tombol kontrol, panel tampilan array, dan area teks untuk menampilkan langkah-langkah pengurutan.

Pengguna dapat memasukkan data angka melalui JTextField dengan format dipisahkan menggunakan koma. Setelah tombol “Set Array” ditekan, data akan disimpan ke dalam array dan ditampilkan pada panel menggunakan JLabel. Program juga melakukan validasi input agar hanya angka yang dapat diproses. Setiap elemen array divisualisasikan dalam bentuk kotak sehingga memudahkan pengguna melihat perubahan data saat sorting berlangsung.

Proses Bubble Sort dijalankan secara bertahap melalui tombol “Langkah Selanjutnya”. Program akan membandingkan dua elemen yang berdekatan, kemudian menukarnya jika posisi elemen tidak sesuai. Elemen yang sedang dibandingkan diberi warna tertentu untuk memperjelas proses sorting. Setiap langkah pengurutan dan hasil sementara array dicatat pada JTextArea sehingga pengguna dapat mengikuti proses pengurutan secara detail. Tombol “Reset” digunakan untuk mengembalikan program ke kondisi awal.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting memiliki peranan penting dalam pengolahan data agar lebih teratur dan mudah diproses. Pada praktikum ini dipelajari algoritma Shell Sort, Merge Sort, dan Quick Sort yang masing-masing memiliki cara kerja dan keunggulan berbeda. Shell Sort melakukan pengurutan menggunakan konsep gap, Merge Sort menggunakan metode divide and conquer, sedangkan Quick Sort melakukan pembagian data berdasarkan pivot.

Selain itu, implementasi Bubble Sort berbasis GUI menggunakan Java Swing membantu memvisualisasikan proses sorting secara interaktif sehingga langkah-langkah pengurutan data lebih mudah dipahami. Dengan praktikum ini, pemahaman mengenai konsep, implementasi, dan efisiensi algoritma sorting dalam bahasa pemrograman Java menjadi semakin baik.

3.2 Saran

Sebaiknya praktikum selanjutnya dapat menambahkan lebih banyak variasi algoritma sorting agar mahasiswa dapat membandingkan tingkat efisiensi dan cara kerja setiap algoritma secara lebih mendalam. Selain itu, visualisasi GUI dapat dikembangkan menjadi lebih menarik dan interaktif sehingga memudahkan pengguna memahami proses sorting dengan lebih jelas. Mahasiswa juga disarankan untuk lebih sering berlatih membuat program sorting agar semakin memahami konsep struktur data dan pemrograman Java.

DAFTAR PUSTAKA

- [1] Teknologi Digital, “Metode Sortir Shell dalam C dan Java: Panduan Lengkap,”. [Daring]. Tersedia pada: [Metode Sortir Shell dalam C dan Java: Panduan Lengkap](#) [Diakses: 25-Mei-2026].
- [2] Guru99, “Algoritma Pengurutan Shell dengan CONTOH,”. [Daring]. Tersedia pada: [Algoritma Pengurutan Shell dengan CONTOH](#) [Diakses: 25-Mei-2026].
- [3] Eren, “Cara Kerja Quick Sort: Meningkatkan Efisiensi Pengurutan Data dengan Langkah-Langkah Sederhana,”. [Daring]. Tersedia pada: [Cara Kerja Quick Sort: Meningkatkan Efisiensi Pengurutan Data Dengan Langkah-Langkah Sederhana - Matob](#) [Diakses: 25-Mei-2026].
- [4] Sutiono, “Algoritma Merge Sort: Pengertian, Kelebihan dan Contoh,”. [Daring]. Tersedia pada: [Algoritma Merge Sort: Pengertian, Kelebihan dan Contoh - DosenIT.com](#) [Diakses: 25-Mei-2026].

LAPORAN TUGAS PRAKTIKUM

Struktur Data

Pekan 6 (Double Linked List)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Sherly Sukmadira



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Tugas Praktikum Struktur Data Pekan 8**Topik: Algoritma Sorting (Shell, Quick, dan Merge)****Tema: Mengurutkan Playlist Lagu****Deskripsi Tugas**

Mahasiswa diminta mengimplementasikan salah SATU algoritma sorting untuk mengurutkan data lagu dalam playlist. Pilih salah satu: Shell Sort, Quick Sort, atau Merge Sort. Data lagu disimpan dalam array (bukan linked list). Program harus bisa mengurutkan berdasarkan Judul (A Z) atau Durasi (terpendek ke terpanjang).

Jawaban

1. Kode Program dan Alasan

Kode program untuk tugas pekan 8 dipisahkan menjadi dua class, yaitu class lagu dan class untuk algoritma yang dipilih. Alasan memilih algoritma sorting Shell karena Algoritma Shell Sort bekerja menggunakan konsep gap yang terus diperkecil hingga seluruh elemen array terurut, menjadikannya lebih efisien dibandingkan algoritma pengurutan sederhana dengan kompleksitas waktu rata-rata $O(n \log n)$. Shell Sort juga merupakan pengembangan dari Insertion Sort yang bekerja dengan membandingkan elemen-elemen. Seiring proses pengurutan berlangsung, nilai gap akan diperkecil hingga menjadi 1 sehingga data menjadi terurut secara keseluruhan. Algoritma ini dipilih karena mudah diimplementasikan, tidak memerlukan memori tambahan yang besar, dan memiliki performa yang lebih baik dibandingkan Insertion Sort pada data berukuran sedang.

1.1 Class Lagu_2511533029

Kode program pertama yaitu sebuah class yang mendefinisikan kelas objek lagu pada Java yang digunakan untuk menyimpan dan mengelola data lagu seperti judul, penyanyi, dan durasi. Kelas ini berperan sebagai model data utama yang mendukung proses pengolahan data pada program sorting. Berikut kode program lagu.

```

1. package pekan8_2511533029;
2.
3. public class Lagu_2511533029 {
4.
5.     String judul_3029;
6.     String penyanyi_3029;
7.     int durasi_3029;
8.
9.     public Lagu_2511533029(String judul_3029,String
        penyanyi_3029,int durasi_3029) {
10.
11.         this.judul_3029 = judul_3029;
12.         this.penyanyi_3029 = penyanyi_3029;
13.         this.durasi_3029 = durasi_3029;

```

```
14.     }
15. }
```

Kode program pertama merupakan sebuah kelas bernama `Lagu_2511533029` yang berfungsi sebagai blueprint atau cetak biru untuk menyimpan data setiap lagu dalam playlist. Kelas `Lagu_2511533029` dilengkapi dengan sebuah constructor yang menerima tiga parameter sesuai atribut yang dimilikinya. Constructor tersebut bertugas menginisialisasi nilai atribut objek menggunakan kata kunci `this`, sehingga setiap kali objek lagu baru dibuat, data judul, penyanyi, dan durasi langsung tersimpan ke dalam objek tersebut.

Kelas ini dirancang secara sederhana tanpa method tambahan karena fungsinya hanya sebagai wadah penyimpanan data. Seluruh atribut bersifat default (`package-private`) sehingga dapat diakses langsung oleh kelas lain dalam package yang sama, yaitu `Sorting_2511533029`. Dengan adanya kelas `Lagu_2511533029`, program menjadi lebih terstruktur karena data setiap lagu dikelola dalam satu objek yang utuh dan terorganisir.

1.2 Class `Sorting_2511533029`

Kelas `Sorting_2511533029` merupakan kelas utama yang mengimplementasikan algoritma Shell Sort untuk mengurutkan data playlist lagu berdasarkan judul secara alfabetis dari A hingga Z. Program menerima input data lagu secara interaktif dari pengguna menggunakan `Scanner`, kemudian menampilkan data sebelum dan sesudah proses pengurutan berlangsung. Berikut kode program sorting.

```
1. package pekan8_2511533029;
2. import java.util.Scanner;
3.
4. public class Sorting_2511533029 {
5.
6.     static Lagu_2511533029[] dataLagu_3029 = new
       Lagu_2511533029[20];
7.     static int jumlahLagu_3029 = 0;
8.     static Scanner scanner_3029 = new Scanner(System.in);
```

```

9.     static void inputData_3029() {
10.         System.out.print("Masukkan jumlah lagu (min 7, maks
11.         20): ");
12.         int n_3029 =
13.         Integer.parseInt(scanner_3029.nextLine().trim());
14.         if (n_3029 < 7)
15.             n_3029 = 7;
16.         if (n_3029 > 20)
17.             n_3029 = 20;
18.         jumlahLagu_3029 = n_3029;
19.         System.out.println();
20.         for (int i_3029 = 0; i_3029 < jumlahLagu_3029;
21.             i_3029++) {
22.             System.out.println( "Lagu ke-" + (i_3029 +
23.             1) );
24.             System.out.print("Judul : ");
25.             String judul_3029 = scanner_3029.nextLine();
26.             System.out.print("Penyanyi : ");
27.             String penyanyi_3029 = scanner_3029.nextLine();
28.             System.out.print("Durasi (detik) : ");
29.             int durasi_3029 =
30.             Integer.parseInt(scanner_3029.nextLine().trim());
31.             dataLagu_3029[i_3029] = new
32.             Lagu_2511533029(judul_3029, penyanyi_3029,durasi_3029);
33.             System.out.println();
34.         }
35.     }
36.     static void shellSort_3029() {
37.         int gap_3029 = jumlahLagu_3029 / 2;
38.         while (gap_3029 > 0) {
39.             for (int i_3029 = gap_3029; i_3029 <
40.                 jumlahLagu_3029; i_3029++) {
41.                 Lagu_2511533029 temp_3029 =
42.                 dataLagu_3029[i_3029];
43.                 int j_3029 = i_3029;
44.                 while (j_3029 >= gap_3029&&
45.                     dataLagu_3029[j_3029 - gap_3029].judul_3029
46.                     .compareToIgnoreCase(temp_3029.judul_3029) > 0) {
47.                     dataLagu_3029[j_3029] =
48.                     dataLagu_3029[j_3029 - gap_3029];
49.                     j_3029 -= gap_3029;
50.                 }
51.                 dataLagu_3029[j_3029] = temp_3029;
52.             }
53.             gap_3029 /= 2;
54.         }
55.     }
56.     static void tampilData_3029(String label_3029) {
57.         System.out.println(label_3029);
58.         for (int i_3029 = 0; i_3029 < jumlahLagu_3029;
59.             i_3029++) {

```

```

50.         Lagu_2511533029 lagu_3029 =
           dataLagu_3029[i_3029];
51.         System.out.printf("%2d. %-30s - %-20s - %d
           detik%n",i_3029 + 1,lagu_3029.judul_3029,
52.         lagu_3029.penyanyi_3029,lagu_3029.durasi_3029);
53.     }
54.     System.out.println();
55. }
56.
57.     public static void main(String[] args_3029) {
58.
59.         System.out.println("=== Sorting Playlist NIM:
           2511533029 ===");
60.         System.out.println("             === SHELL SORT ===");
61.         System.out.println();
62.
63.         inputData_3029();
64.         tampilData_3029("— Data Sebelum Shell Sort —");
65.         shellSort_3029();
66.         tampilData_3029("— Data Setelah Shell Sort (Judul
           A-Z) —");
67.         scanner_3029.close();
68.     }
69. }

```

Kode program kedua merupakan kelas utama bernama `Sorting_2511533029` yang bertugas mengelola keseluruhan proses pengurutan data lagu menggunakan algoritma Shell Sort. Kelas ini mendeklarasikan sebuah array statis `dataLagu_3029` bertipe `Lagu_2511533029` dengan kapasitas maksimal 20 elemen, variabel `jumlahLagu_3029` untuk mencatat jumlah lagu yang diinput, serta objek Scanner bernama `scanner_3029` untuk membaca input dari pengguna melalui keyboard. Ketiga elemen tersebut dideklarasikan sebagai static agar dapat diakses oleh seluruh method dalam kelas tanpa perlu membuat objek terlebih dahulu.

Method `inputData_3029()` berfungsi untuk menerima input data lagu dari pengguna secara interaktif menggunakan Scanner. Pengguna diminta memasukkan jumlah lagu terlebih dahulu dengan batas minimal 7 dan maksimal 20, kemudian program akan meminta input judul, penyanyi, dan durasi untuk setiap lagu secara berurutan. Setiap data yang dimasukkan

kemudian disimpan sebagai objek `Lagu_2511533029` ke dalam array `dataLagu_3029`. Sementara itu, method `tampilData_3029()` bertugas menampilkan seluruh isi array ke layar dalam format yang rapi menggunakan `System.out.printf`, sehingga data lagu terlihat terstruktur dengan kolom judul, penyanyi, dan durasi yang tersusun dengan baik.

Method `shellSort_3029()` merupakan inti dari program ini, yaitu implementasi algoritma Shell Sort untuk mengurutkan data lagu berdasarkan judul secara alfabetis dari A hingga Z. Proses pengurutan dimulai dengan menentukan nilai gap sebesar setengah dari jumlah lagu, kemudian setiap iterasi gap dibagi dua hingga bernilai nol. Pada setiap nilai gap, elemen-elemen array dibandingkan menggunakan method `compareToIgnoreCase()` agar perbandingan judul tidak terpengaruh oleh huruf kapital atau huruf kecil. Pada method `main`, program menampilkan header identitas, memanggil `inputData_3029()` untuk mengisi data, menampilkan data sebelum diurutkan, menjalankan `shellSort_3029()`, lalu menampilkan kembali data setelah proses pengurutan selesai.

2. Input dan Output

Berikut merupakan input yang saya buat ketika kode program dijalankan serta output yang akan dihasilkan.

```

=== Sorting Playlist NIM: 2511533029 ===
=== SHELL SORT ===

Masukkan jumlah lagu (min 7, maks 20): 7

Lagu ke-1
Judul : Cantik
Penyanyi : Kahitna
Durasi (detik) : 245

Lagu ke-2
Judul : Andai Dia Tahu
Penyanyi : Kahitna
Durasi (detik) : 278

Lagu ke-3
Judul : Cerita Cinta
Penyanyi : Kahitna
Durasi (detik) : 265

Lagu ke-4
Judul : My Love
Penyanyi : Westlife
Durasi (detik) : 232

Lagu ke-5
Judul : If I Let You Go
Penyanyi : Westlife
Durasi (detik) : 223

Lagu ke-6
Judul : Swear It Again
Penyanyi : Westlife
Durasi (detik) : 241

Lagu ke-7
Judul : Flying Without Wings
Penyanyi : Westlife
Durasi (detik) : 220

```

```

— Data Sebelum Shell Sort —
1. Cantik - Kahitna - 245 detik
2. Andai Dia Tahu - Kahitna - 278 detik
3. Cerita Cinta - Kahitna - 265 detik
4. My Love - Westlife - 232 detik
5. If I Let You Go - Westlife - 223 detik
6. Swear It Again - Westlife - 241 detik
7. Flying Without Wings - Westlife - 220 detik

— Data Setelah Shell Sort (Judul A-Z) —
1. Andai Dia Tahu - Kahitna - 278 detik
2. Cantik - Kahitna - 245 detik
3. Cerita Cinta - Kahitna - 265 detik
4. Flying Without Wings - Westlife - 220 detik
5. If I Let You Go - Westlife - 223 detik
6. My Love - Westlife - 232 detik
7. Swear It Again - Westlife - 241 detik

```